



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems Lecture 20

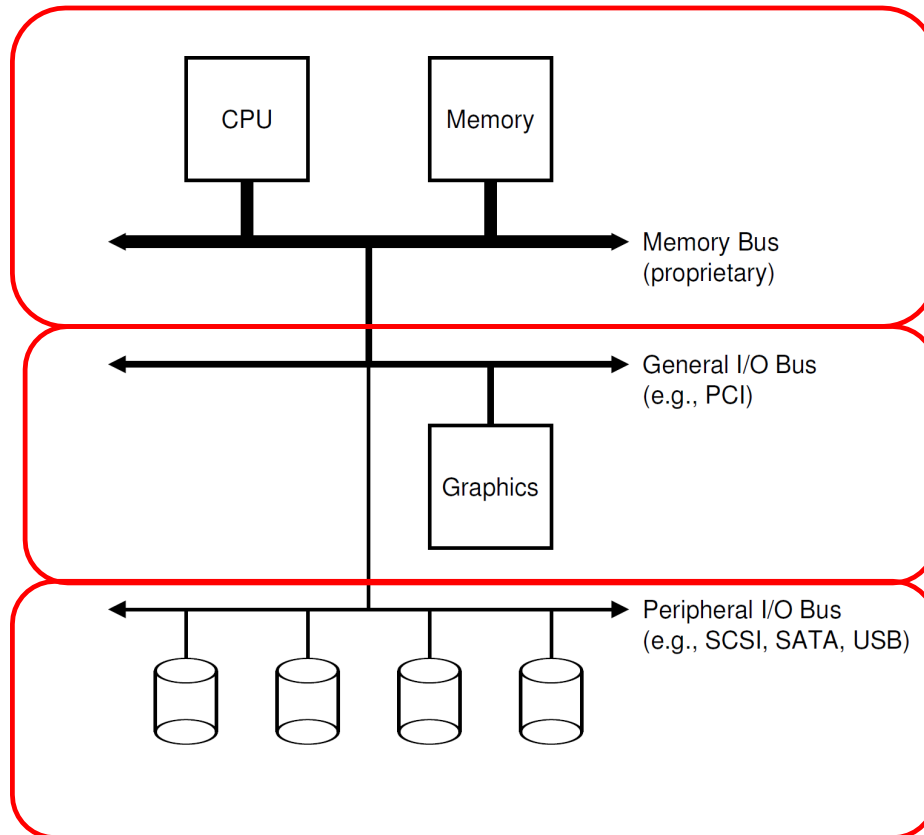
File System 1: I/O & storage devices

**Prof. Yunming XIAO
School of Data Science**

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

I/O basics

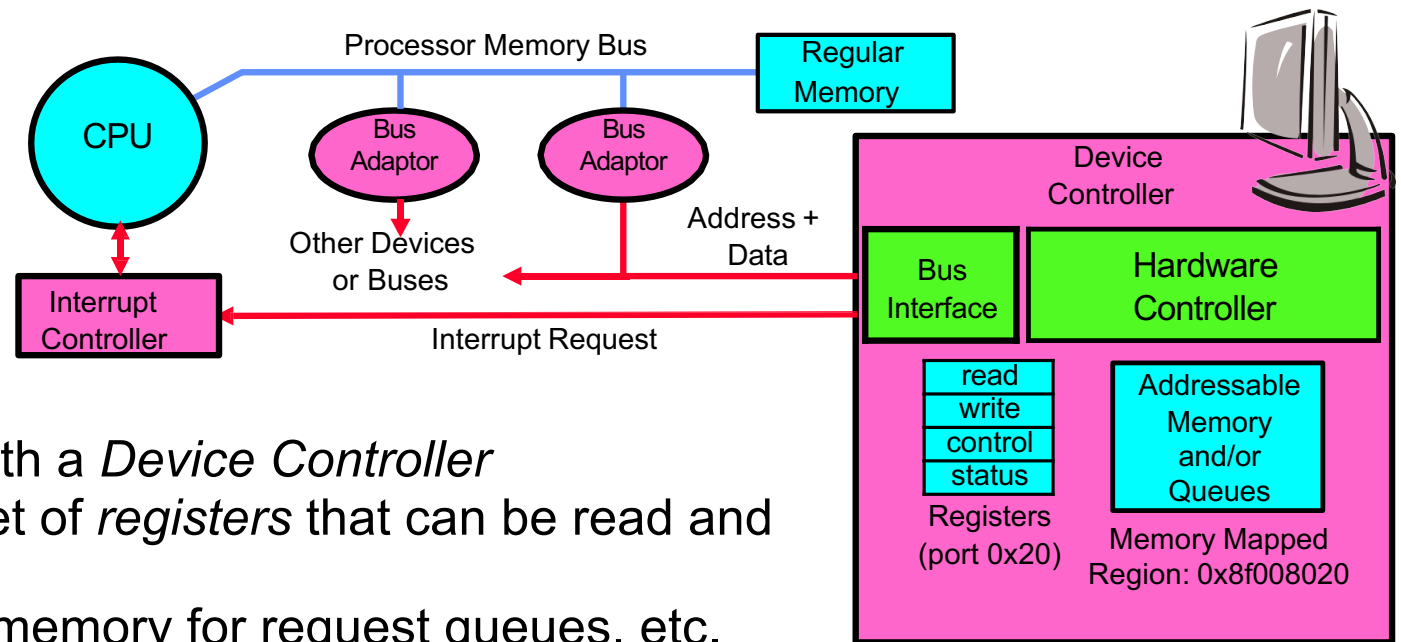
Simplified IO architecture



Follows a hierarchical structure
because of cost:

the faster the bus, the more
expensive it is

How does the processor talk to devices?

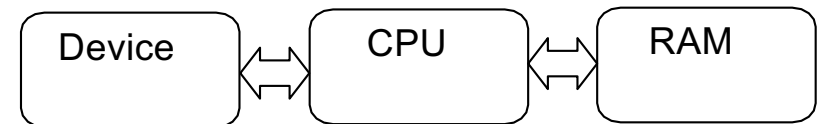


- CPU interacts with a *Device Controller*
 - Contains a set of *registers* that can be read and written
 - May contain memory for request queues, etc.
- Processor accesses registers in two ways:
 - **Port-Mapped I/O**: in/out instructions
 - Example from the Intel architecture: out 0x21,AL
 - **Memory-mapped I/O**: load/store instructions
 - Registers/memory appear in physical address space
 - I/O accomplished with standard load & store instructions

Programmed I/O vs. direct memory access

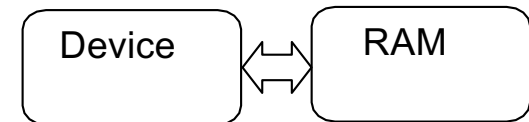
- With programmed I/O (our simple protocol):

- CPU issues read request
- Device interrupts CPU with data
- CPU writes data to memory
- Pros: simple hardware. Cons: Poor CPU is always busy!



- With direct-memory-access (DMA):

- CPU sets up DMA request
 - Gives controller access to memory bus
- Device puts data on bus & RAM accepts it
- Device interrupts CPU when done



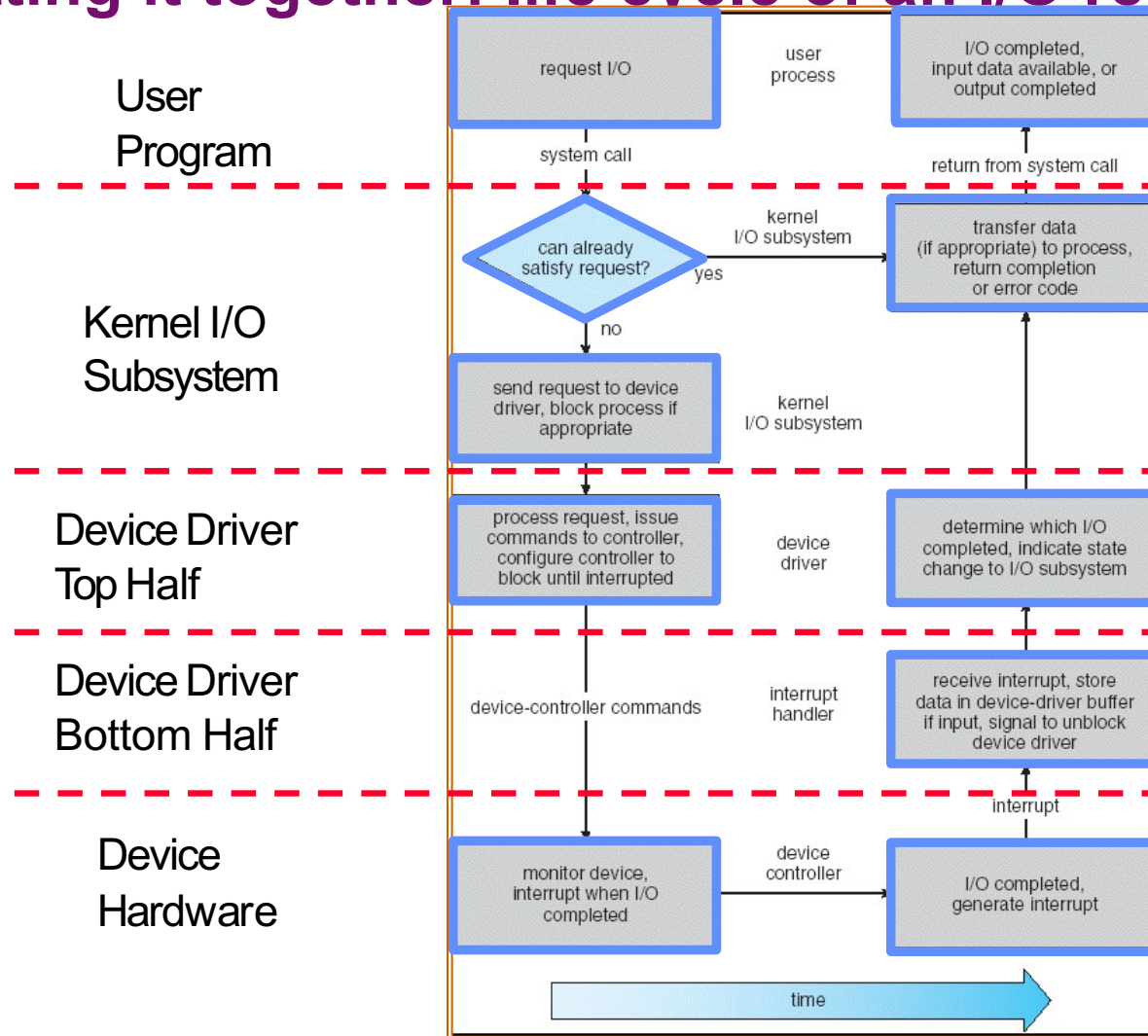
How can the OS handle ~~one~~ all devices

- How do we fit devices with specific interfaces into the OS, which should remain general?
 - Build a “device-neutral” OS and hide details of devices from most of OS
- Abstraction to the rescue!
 - **Device Drivers** encapsulate all specifics of device interaction
 - Implement device-neutral interfaces

Device Drivers

- **Device Driver:** Device-specific code in the kernel that interacts directly with that device hardware
 - Supports a standard, internal interface
 - Special device-specific configuration supported with the `ioctl()` system call
- Device Drivers are typically divided into two pieces:
 - **Top half:** accessed in call path from system calls
 - implements a set of standard, cross-device calls like `open()`, `close()`, `read()`, `write()`, `ioctl()`, `strategy()`
 - This is the kernel's interface to the device driver
 - Top half will *start* I/O to device, may put thread to sleep until finished
 - **Bottom half:** run as interrupt routine
 - Gets input or transfers next block of output
 - May wake sleeping threads if I/O now complete
- Your body is 90% water, your OS is 70% device-drivers

Putting it together: life cycle of an I/O request



Summary

- I/O Devices Types:
 - Many different speeds (0.1 bytes/sec to 50+ GBytes/sec)
 - Different Access Patterns:
 - Block Devices, Character Devices, Network Devices
- Device Controllers: Hardware that controls actual device
 - Processor Accesses through I/O instructions (port-mapped I/O) or load/store to special physical memory addresses (memory-mapped I/O)
- Notification mechanisms
 - Interrupts
 - Polling: Report results through status register that processor looks at periodically
- Device drivers interface to I/O devices
 - Provide clean Read/Write interface to OS above
 - Manipulate devices through PIO, DMA & interrupt handling

Hard Disk Driver (HDD)

How does user deal with timing?

- **Blocking Interface:** “Wait”
 - When request data (e.g. `read()` system call), put process to sleep until data is ready
 - When write data (e.g. `write()` system call), put process to sleep
 - Until device is ready for data
- **Non-blocking Interface:** “Don’t Wait”
 - Returns quickly from read or write request with count of bytes successfully transferred
 - Read may return nothing, write may write nothing
- **Asynchronous Interface:** “Tell Me Later”
 - When request data, take pointer to user’s buffer, return immediately; later kernel fills buffer and notifies user (or lets user check/poll)
 - When send data, take pointer to user’s buffer, return immediately; later kernel takes data and notifies user (or lets user check/poll)

Storage and file systems

- Storage devices: how do we measure their performance and how do they work?
 - We'll focus on hard disks and solid-state drives (SSDs)
- File systems: most common abstraction on top of storage devices
 - Let users organize data into named files in hierarchical paths

Measuring storage device performance

- *Latency*: time to complete a task
 - Measured in units of time (s, ms, us, ..., hours, years)
- *Throughput* or *Bandwidth*: rate at which tasks are performed
 - Measured in units of things per unit time (bytes/s, ops/s)
- *Start up* or *Overhead*: time to initiate an operation
 - Most I/O operations are roughly linear in b bytes
 - $\text{Latency}(b) = \text{Overhead} + b/\text{TransferCapacity}$

Storage devices

- Magnetic disks

- Storage that rarely becomes corrupted
- Large capacity at low cost
- Block level random access (except for SMR – later!)
- Slow performance for random access
- Better performance for sequential access

- Flash memory

- Storage that rarely becomes corrupted
- Capacity at intermediate cost (5-20x disk)
- Block level random access
- Good performance for reads; worse for random writes
- Wear patterns issue

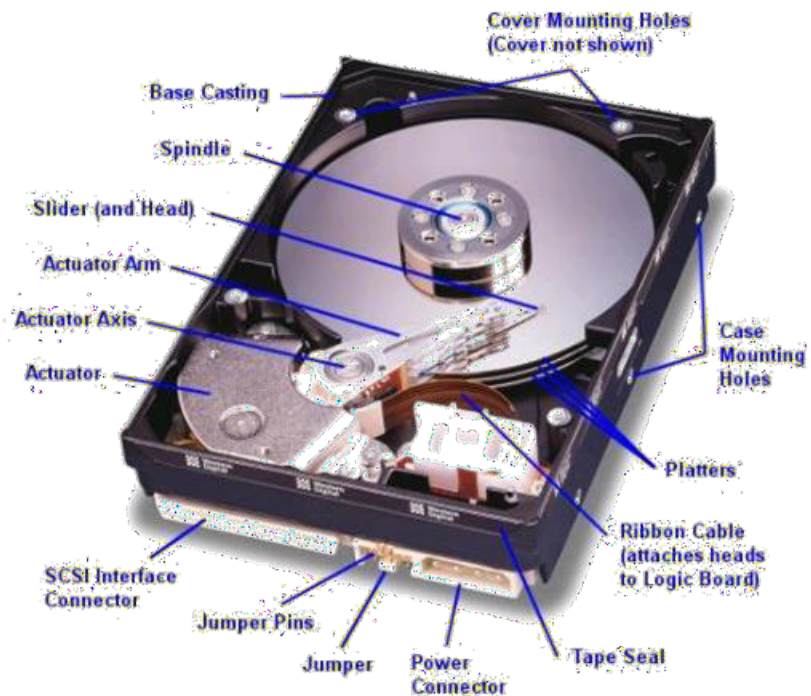
Hard Disk Drives (HDDs)



IBM Model 350, 1956
(first commercial HDD, 3.75 MB)



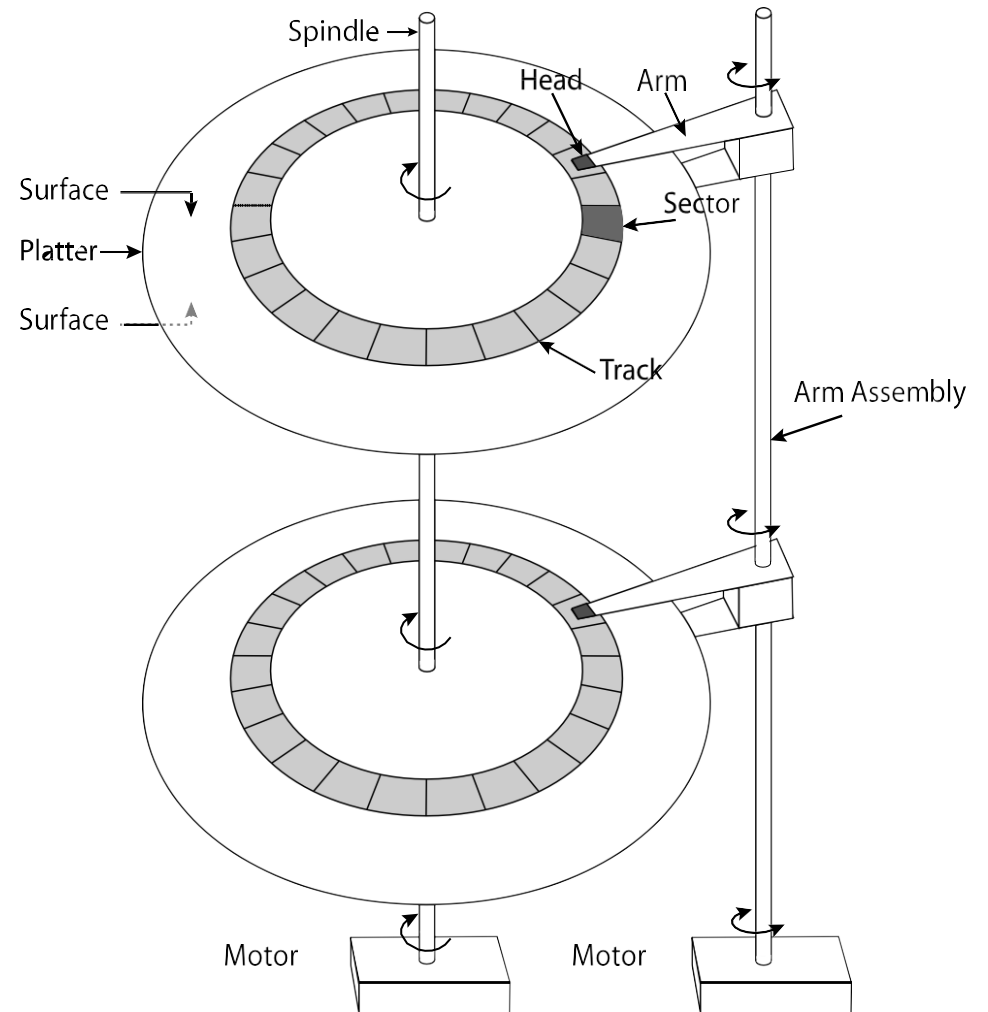
Hard Disk Drives (HDDs)



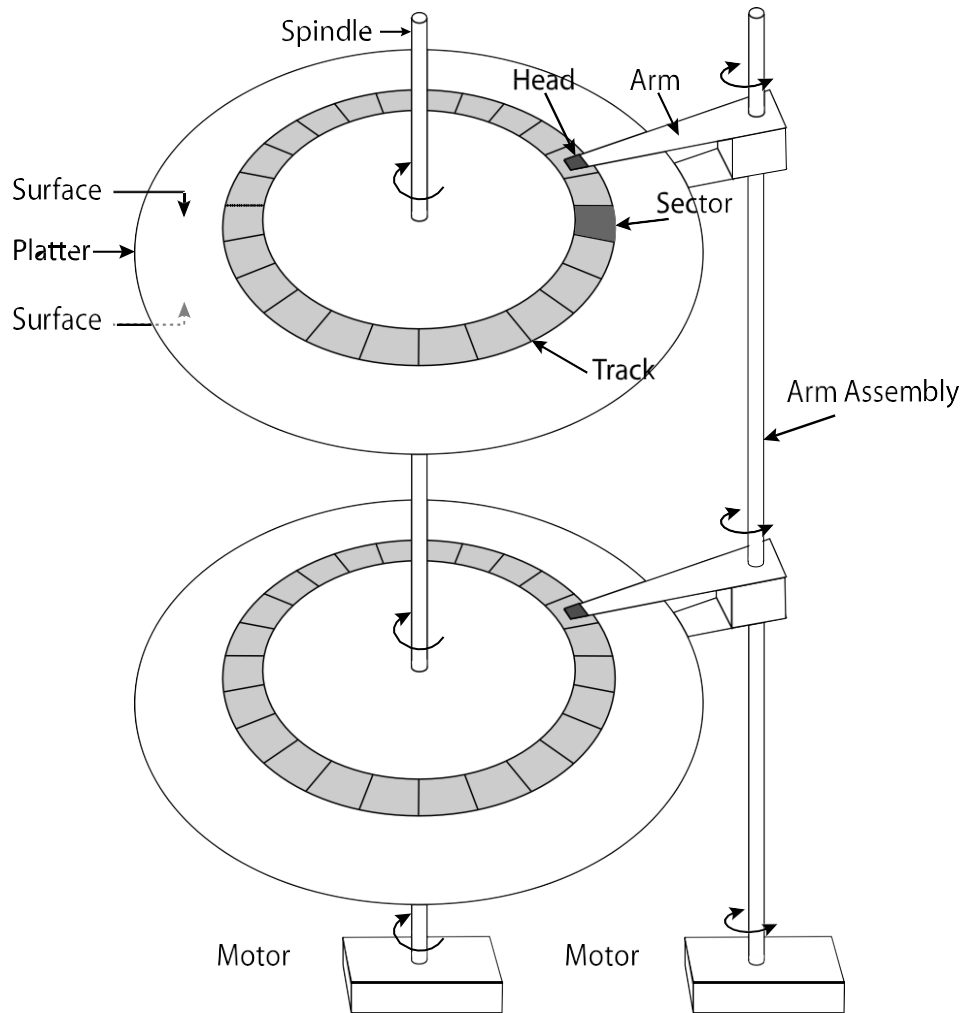
Read/Write Head Side View

The amazing magnetic disk

Store data magnetically on thin metallic film bonded to rotating disk of glass, ceramic, or aluminum

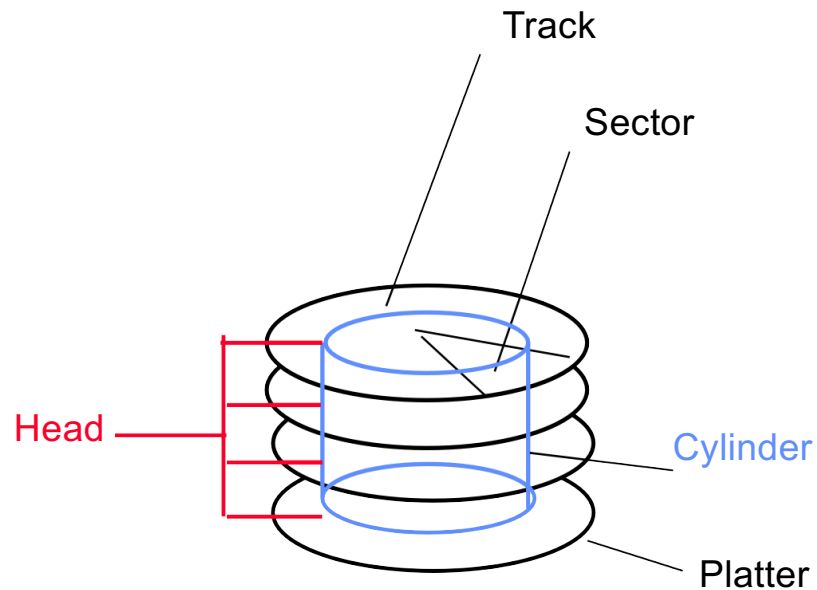


The amazing magnetic disk



- **Track:** concentric circle on surface
- **Sectors:** slice of a track
 - Smallest addressable unit
 - Are units of transfers
- **Cylinder:** all the tracks under the head at a given point, across all surfaces

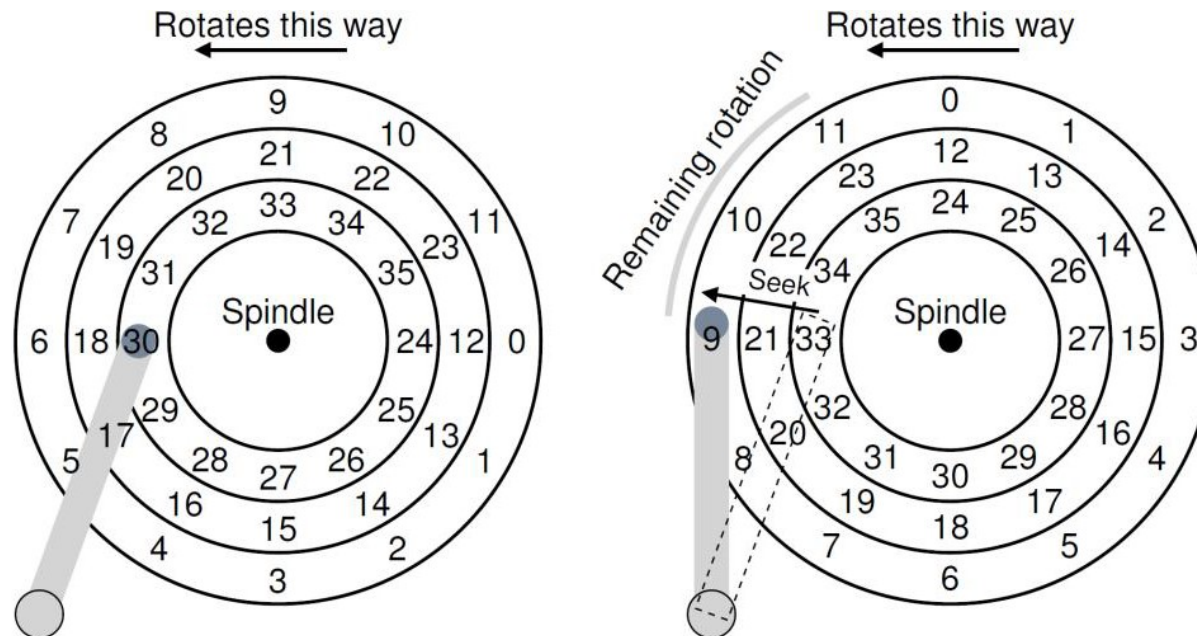
The amazing magnetic disk



- Track lengths vary across disk: outside tracks have more sectors per track, higher bandwidth
- Disk is organized into regions of tracks with the same number of sector/tracks
- Usually, only outer half of radius is used

Reading/Writing data

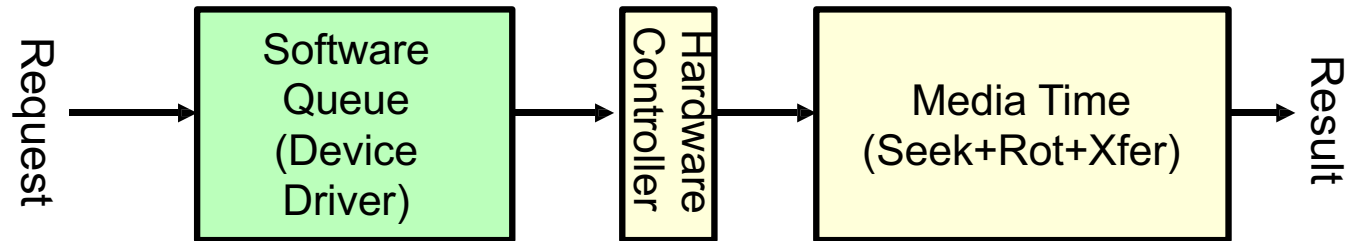
- **Seek time:** position the head/arm over the proper track
- **Rotational latency:** wait for desired sector to rotate under r/w head
- **Transfer time:** transfer a block of bits (sector) under r/w head



Reading/Writing data

Request Time =

Queueing Time + Controller Time + Seek + Rotational + Transfer



Typical numbers for magnetic disk

Parameter	Info/Range
Space/Density	Space: 18TB (Seagate), 9 platters, in 3½ inch form factor! Areal Density: ≥ 1 Terabit/square inch! (PMR, Helium, ...)
Average Seek Time	Typically 4-6 milliseconds
Average Rotational Latency	Most laptop/desktop disks rotate at 3600-7200 RPM (16-8 ms/rotation). Server disks up to 15,000 RPM. Average latency is halfway around disk so 4-8 milliseconds
Controller Time	Depends on controller hardware
Transfer Time	Typically 50 to 250 MB/s. Depends on: • Transfer size (usually a sector): 512B – 1KB per sector • Rotation speed: 3600 RPM to 15000 RPM • Recording density: bits per inch on a track • Diameter: ranges from 1 in to 5.25 in
Cost	Used to drop by about 2x every 1.5 years, now slowing down

Disk performance example

- Key to using disk effectively (especially for file systems) is to minimize seek & rotational delays
- Example:
 - Assume a disk with avg seek time of 5ms, 7200RPM $\Rightarrow$
Time for rotation: $60000 \text{ (ms/min)} / 7200 \text{ (rev/min)} \approx 8\text{ms}$
 - Assume transfer rate of 50MByte/s, block size of 4Kbyte $\Rightarrow$
Time to read 1 sector: $4096 \text{ bytes} / 50 \times 10^6 \text{ (bytes/s)} = 81.92 \times 10^{-6} \text{ sec} \approx 0.082\text{ms}$

Do access patterns influence how fast can read/write to disk?

Disk performance example

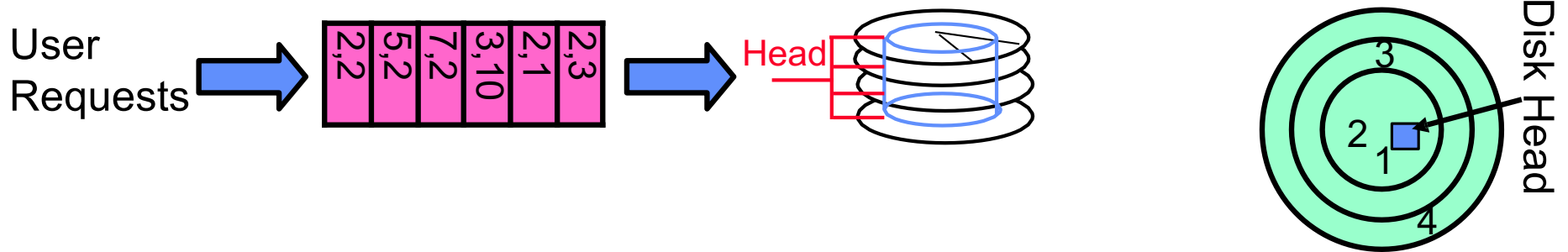
- Read block from **random place on disk** (random reads):
 - Seek (5ms) + Rot. Delay (4ms) + Transfer (0.082ms) = 9.082ms
 - Approx 9ms to fetch/put data: $4096 \text{ bytes} / 9.082 \times 10^{-3} \text{ s} \Rightarrow 451 \text{ KB/s}$
- Read block from **random place in same cylinder**:
 - Rot. Delay (4ms) + Transfer (0.082ms) = 4.082ms
 - Approx 4ms to fetch/put data: $4096 \text{ bytes} / 4.082 \times 10^{-3} \text{ s} \Rightarrow 1.03 \text{ MB/s}$
- Read next block **on same track** (sequential reads):
 - Transfer (0.082ms): $4096 \text{ bytes} / 0.082 \times 10^{-3} \text{ s} \Rightarrow 50 \text{ MB/sec}$

When is disk performance highest?

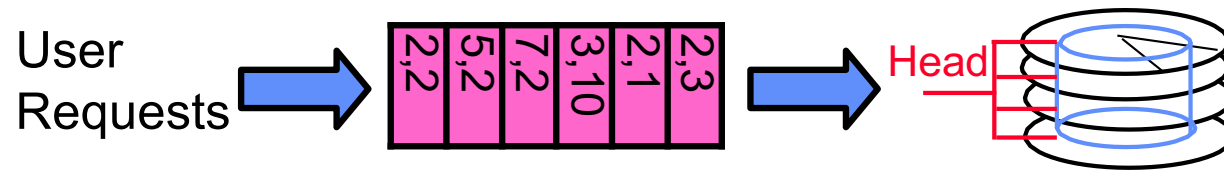
- When there are big sequential reads, or
- When there is so much work to do that they can be piggy backed (reordering queues—one moment)
- OK to be inefficient when things are mostly idle
- Bursts are both a threat and an opportunity
- Waste space for speed?

Disk scheduling (1/3)

- Disk can do only one request at a time; What order do you choose to do queued requests?

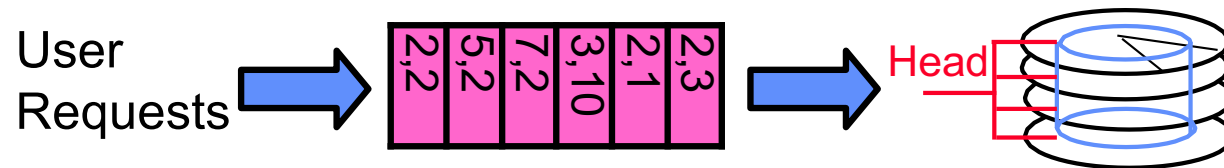


Disk scheduling (1/3)

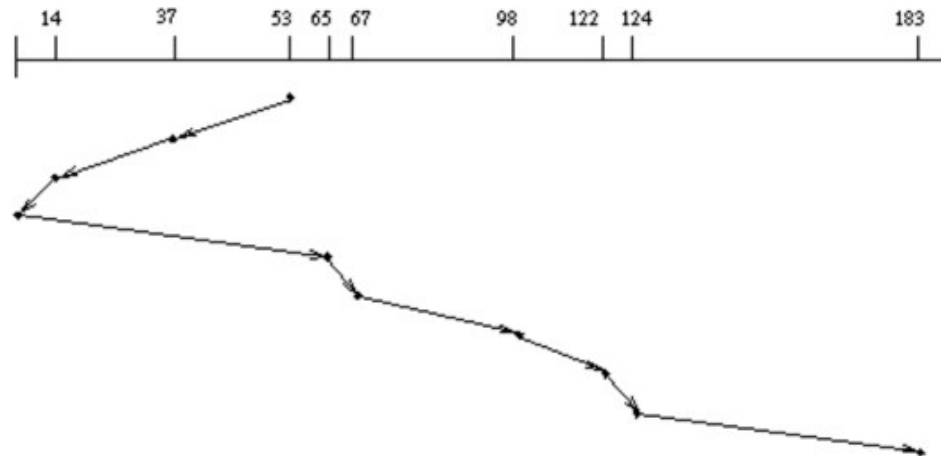


- FIFO order
 - Fair among requesters, but order of arrival may be to random spots on the disk
- SSTF: Shortest seek time first
 - Pick the request that's closest on the disk
 - Con: SSTF good at reducing seeks, but may lead to starvation

Disk scheduling (2/3)

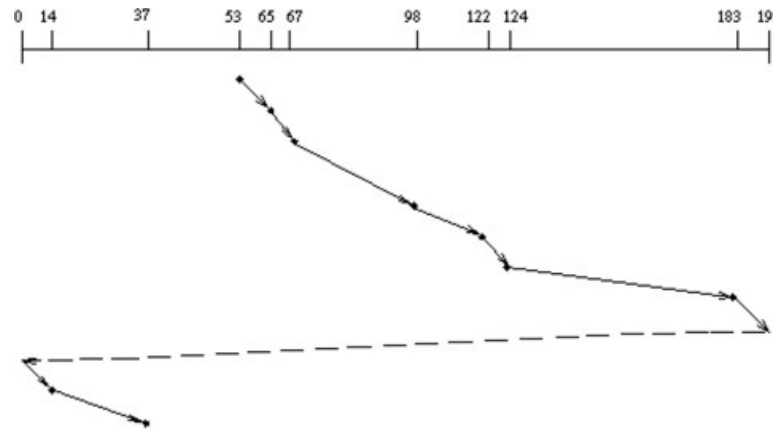


- SCAN: Implements an Elevator Algorithm: take the closest request in the direction of travel
 - No starvation, but retains flavor of SSTF



Disk scheduling (3/3)

- C-SCAN: Circular-Scan: only goes in one direction
 - Skips any requests on the way back
 - Fairer than SCAN, not biased towards pages in middle



Lots of intelligence in the device controller

- **Shingled Magnetic Recording (SMR):** pack more data with error correcting codes
 - Disk write head has wider field than read head, so overlap tracks
 - Hide corruptions due to neighboring track writes (but must fix in background)
- **Sector sparing:** remap bad sectors transparently to spare sectors on the same surface
- **Slip sparing:** remap all sectors (when there is a bad sector) to preserve sequential behavior
- **Track skewing:** sector numbers offset from one track to the next, to allow for disk head movement for sequential ops

Example of current HDD

- Seagate Exos X24 (2023)

- 24 TB hard disk
 - 10 platters, 20 heads
 - 1.26 TB/in²
 - Helium filled: reduce friction and power
- 4.16 ms average seek time
- 4096 byte physical sectors
- 7200 RPMs
- Dual 6 Gbps SATA /12Gbps SAS interface
 - 285MB/s MAX transfer rate
 - Cache size: 512MB
- Price: \$ 479 (~ \$0.02/GB)

800kx



- Original IBM Personal Computer/AT (1986)

- 30 MB hard disk
- 30-40 ms average seek time
- 0.7-1 MB/s (est.)
- Price: \$500 (\$17K/GB)

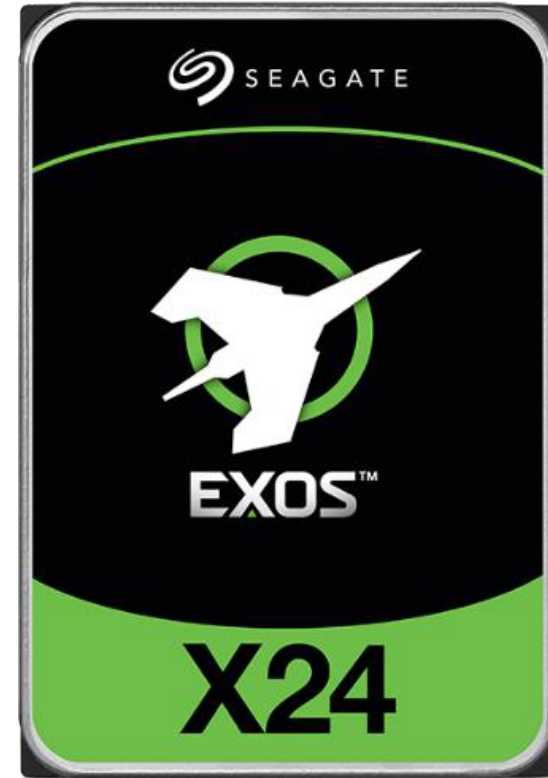
385x



850kx



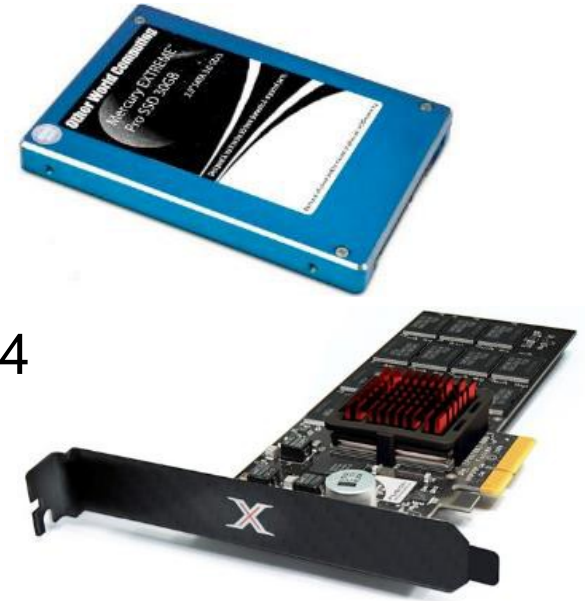
10x



Solid State Driver (SSD)

Solid State Drives (SSDs)

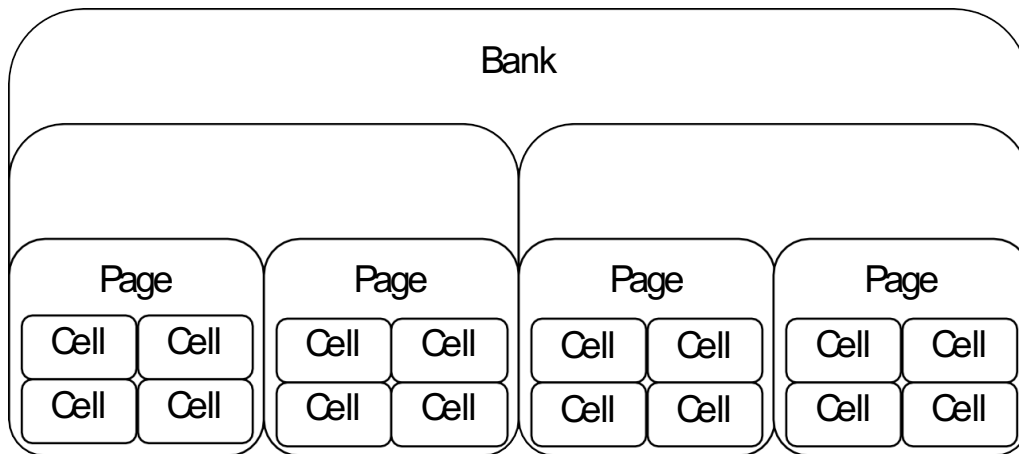
- Invented by Fujio Masuoka at Toshiba in 1980
- Modern flash memory in laptops, phones, etc.:
 - Sector (4 KB page) addressable, but stores 4-64 “pages” per memory “block”
 - Trapped electrons distinguish between 1 and 0
- No moving parts (no rotate/seek motors)
 - Eliminates seek and rotational delay (0.1-0.2ms access time)
 - Very low power and lightweight
 - Limited “write cycles”



The flash cell

- Encode bit by trapping electrons into a cell
- Single-level cell (SLC)
 - Single bit is stored within a transistor
 - Faster, more lasting (50k to 100k writes before wear out)
- Multi-level cell (MLC)
 - Two/three bits are encoded into different levels of charge
 - Wear out much faster (1k to 10k writes)

Of banks, blocks, and cells



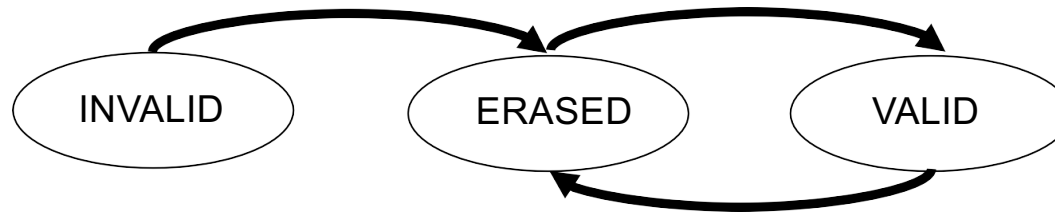
- Flash chips organized in **banks**
 - Banks can be accessed in parallel
- **Blocks:** 128 KB/256KB
 - (64 to 258 pages)
- **Pages:** Few KB
- **Cells:** 1 to 4 bits
- Distinction between blocks and pages important in operations!

⚠ Confusing:
not same as OS
and HDD "blocks"!

Low-level flash operations

- How do you **read**?
 - Chip supports reading pages
 - 10s of microseconds, independently of the previously read page
- What about **writing**? More complicated!
 - Must first *erase* the block
 - Erase quite expensive (milliseconds)
 - Once block has been erased, can then *program* a page
 - Change 1s to 0s within a page.
 - 100s of microseconds.
 - Blocks can only be erased a limited number of times!

Low-level flash operations



		<i>i i i i</i>	<i>Initial: pages in block are invalid (i)</i>
Erase()	→	<i>E E E E</i>	<i>State of pages in block set to erased (E)</i>
Program(0)	→	<i>V E E E</i>	<i>Program page 0; state set to valid (V)</i>
Program(0)	→	error	<i>Cannot re-program page after programming</i>
Program(1)	→	<i>V V E E</i>	<i>Program page 1</i>
Erase()	→	<i>E E E E</i>	<i>Contents erased; all pages programmable</i>

Low-level flash operations

- Assume block of 4 pages. All valid.
- Want to write Page 0

Page 0	Page 1	Page 2	Page 3
00011000	11001110	00000001	00111111
VALID	VALID	VALID	VALID

Step 1: erase full block

Page 0	Page 1	Page 2	Page 3
11111111	11111111	11111111	11111111
ERASED	ERASED	ERASED	ERASED

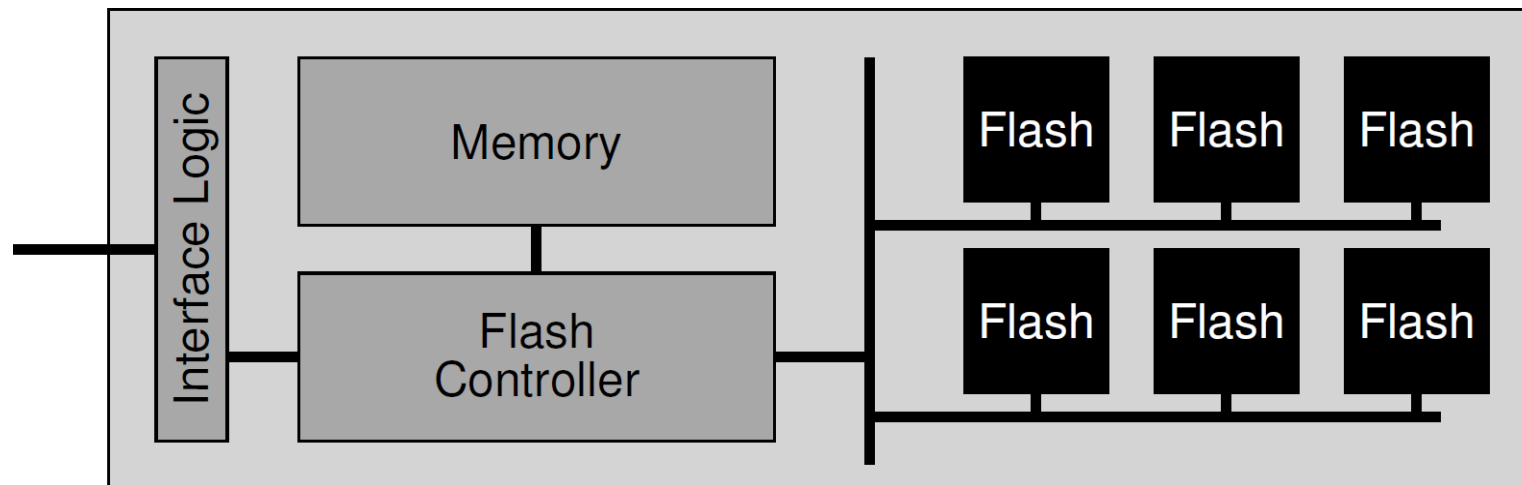
Step 2: program page 0

Page 0	Page 1	Page 2	Page 3
00000011	11111111	11111111	11111111
VALID	ERASED	ERASED	ERASED

SSD architecture

- Recall that SSDs use low-level Flash operations to provide same interface as HDD
 - read and write chunk (4KB) at a time
- Reads are easy, but for writes, can only overwrite data one block (256KB) at a time!
- Why not just erase and rewrite new version of entire 256KB block?
 - Erasure is very slow (milliseconds)
 - Each block has a finite lifetime, can only be erased and rewritten about 10K times
 - Heavily used blocks likely to wear out quickly

SSD architecture (simplified)



Flash Translation Layer (FTL)

- Add a layer of indirection: the flash translation layer
 - Translates request for logical blocks (device interface) to low-level Flash blocks and pages
- Reduce write amplification
 - Ratio of the total write traffic in bytes issues by the flash chip by the FTL divided by the total write traffic issued by the OS to the device
- Avoid wear out:
 - A single block should not be erased too often

FTL – two systems principles

- FTL uses *indirection* and *copy-on-write*
- Maintains mapping tables in DRAM
 - Map virtual block numbers (which OS uses) to physical page numbers (which flash memory controller uses)
 - Can now freely relocate data w/o OS knowing
- Copy on Write/ Log-structured FTL
 - Don't overwrite a page when OS updates its data
 - Instead, write new version in a free page
 - Update FTL mapping to point to new location

FTL Example

Initial
State

Mapping Table:			
Block 0		Block 1	
E	E	E	E

Write(a0)

Mapping Table: a0->0,0			
Block 0		Block 1	
a0			
V	E	E	E

Write(a1)

Mapping Table: a0->0,0/a1->0,1			
Block 0		Block 1	
a0	a1		
V	V	E	E

Write(a1)

Mapping Table: a0->0,0/a1->1,0			
Block 0		Block 1	
a0	a1	a1	
V	V	V	E

Write(a0)

Mapping Table: a0->1,1/a1->1,0			
Block 0		Block 1	
a0	a1	a1	a0
V	V	V	V

Garbage
Collect

Mapping Table: a0->1,1/a1->1,0			
Block 0		Block 1	
		a1	a0
E	E	V	V

Examples of recent SSDs

- Silicon Power 4TB **SATA** Internal SSD (2023)
 - Seq reads 540 MB/s
 - Seq writes 500 MB/s
 - Price (Amazon): \$184 (\$0.046/GB)



- Micron 9300 Pro 15TB **NVMe (PCIe)** Enterprise SSD (2019)
 - Seq reads/writes: 3500 MB/s
 - Random Read Ops (IOPS): 100K+
 - Price: \$1750 (\$0.11/GB)



HDD vs. SSD Comparison

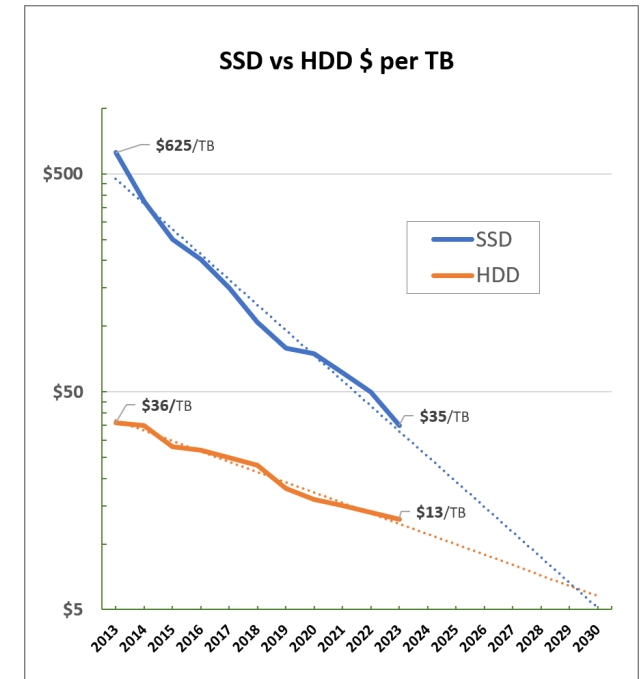


SSD vs HDD

Usually 10 000 or 15 000 rpm SAS drives

0.1 ms	Access times SSDs exhibit virtually no access time	5.5 ~ 8.0 ms
SSDs deliver at least 6000 io/s	Random I/O Performance SSDs are at least 15 times faster than HDDs	HDDs reach up to 400 io/s
SSDs have a failure rate of less than 0.5 %	Reliability This makes SSDs 4 - 10 times more reliable	HDD's failure rate fluctuates between 2 ~ 5 %
SSDs consume between 2 & 5 watts	Energy savings This means that on a large server like ours, approximately 100 watts are saved	HDDs consume between 6 & 15 watts
SSDs have an average I/O wait of 1 %	CPU Power You will have an extra 6% of CPU power for other operations	HDDs' average I/O wait is about 7 %
the average service time for an I/O request while running a backup remains below 20 ms	Input/Output request times SSDs allow for much faster data access	the I/O request time with HDDs during backup rises up to 400~500 ms
SSD backups take about 6 hours	Backup Rates SSDs allows for 3 - 5 times faster backups for your data	HDD backups take up to 20~24 hours

HDD	SSD
Require seek + rotation	No seeks
Not parallel (one head)	Parallel
Brittle (moving parts)	No moving parts
Random reads take 10s milliseconds	Random reads take 10s microseconds
Slow (mechanical)	Wears out
Cheap/large storage	Expensive/smaller storage



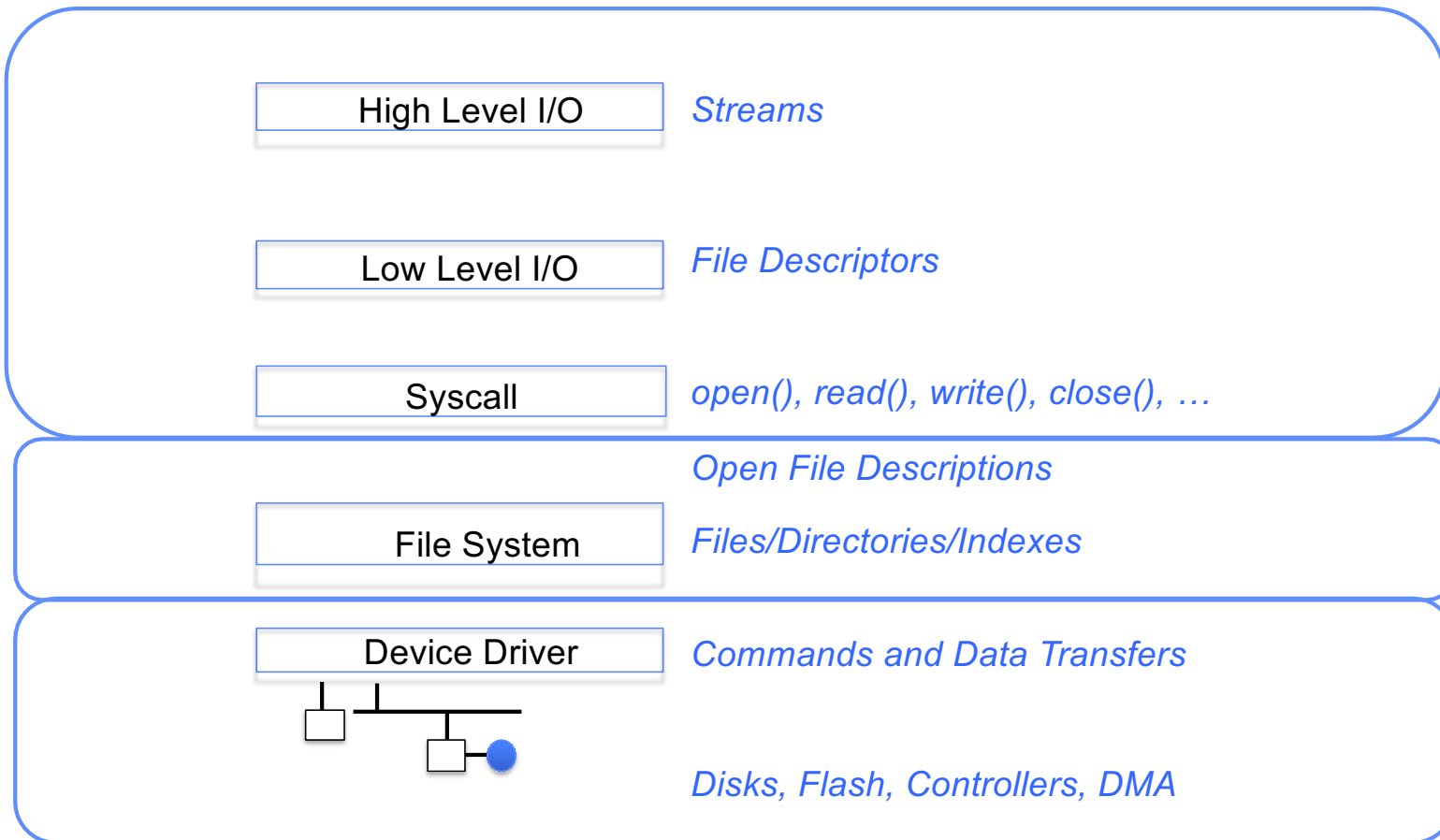
SSD prices falling faster than HDD!

SSD Summary

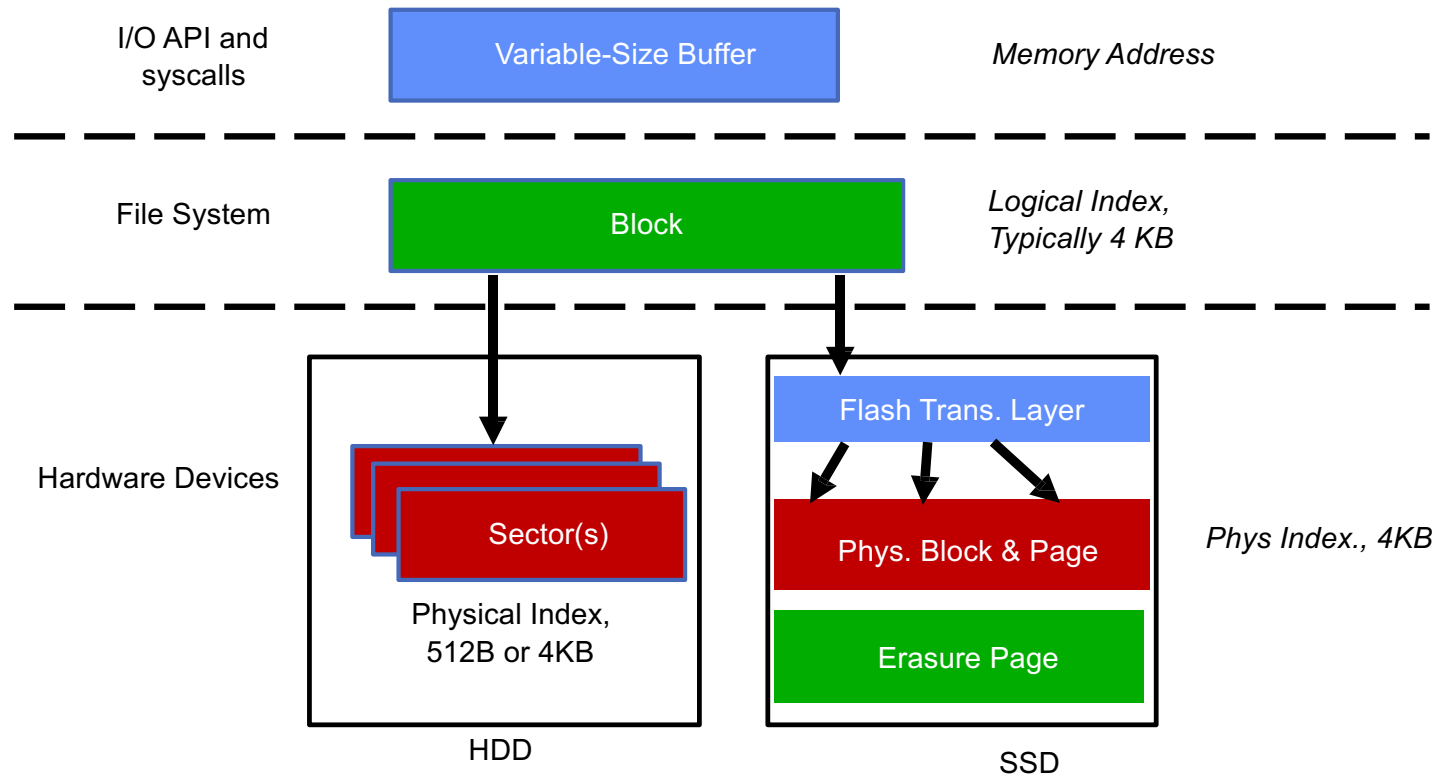
- Pros (vs. hard disk drives):
 - Low latency, high throughput (eliminate seek/rotational delay)
 - No moving parts:
 - Very light weight, low power, silent, very shock insensitive
 - Read at high speeds (limited by controller and I/O bus)
- Cons
 - Expensive (2-10x disk)
 - Asymmetric block write performance: read pg/erase/write pg
 - Controller garbage collection (GC) algorithms have major effect on performance
 - Limited drive lifetime
 - 1-10K writes/page for MLC NAND
 - Avg failure rate is 6 years, life expectancy is 9–11 years
- These are changing rapidly!

File Systems

Recall: I/O and storage layers



From storage to file systems



Building a file system

- Layer of OS that transforms block interface of disks (or other block devices) into Files, Directories, etc.

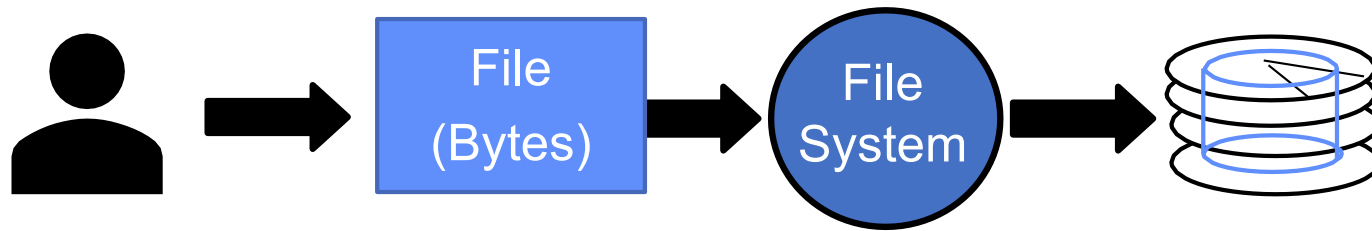
Building a file system

- OS as an illusionist:
 - Take limited hardware interface (array of blocks) and provide a more convenient/useful interface with:
 - Naming:
 - Find file by name, not block numbers
 - Organize file names with directories
 - Organization: Map files to blocks
 - Protection: Enforce access restrictions
 - Reliability: Keep files intact despite crashes, failures, etc.

User vs. System view of a file

- User's view:
 - Durable Data Structures
- System's view (system call interface):
 - Collection of Bytes (UNIX)
 - Doesn't matter to system what kind of data structures you want to store on disk!
- System's view (inside OS):
 - Collection of blocks (a block is a logical transfer unit, while a sector is the physical transfer unit)
 - Block size $\geq$ sector size; in UNIX, block size is 4KB

Translation from user to system view



- What happens if user says: “give me bytes 2 – 12?”
 - Fetch block corresponding to those bytes
 - Return just the correct portion of the block
- What about writing bytes 2 – 12?
 - Fetch block, modify relevant portion, write out block
- Everything inside file system is in terms of whole-size blocks

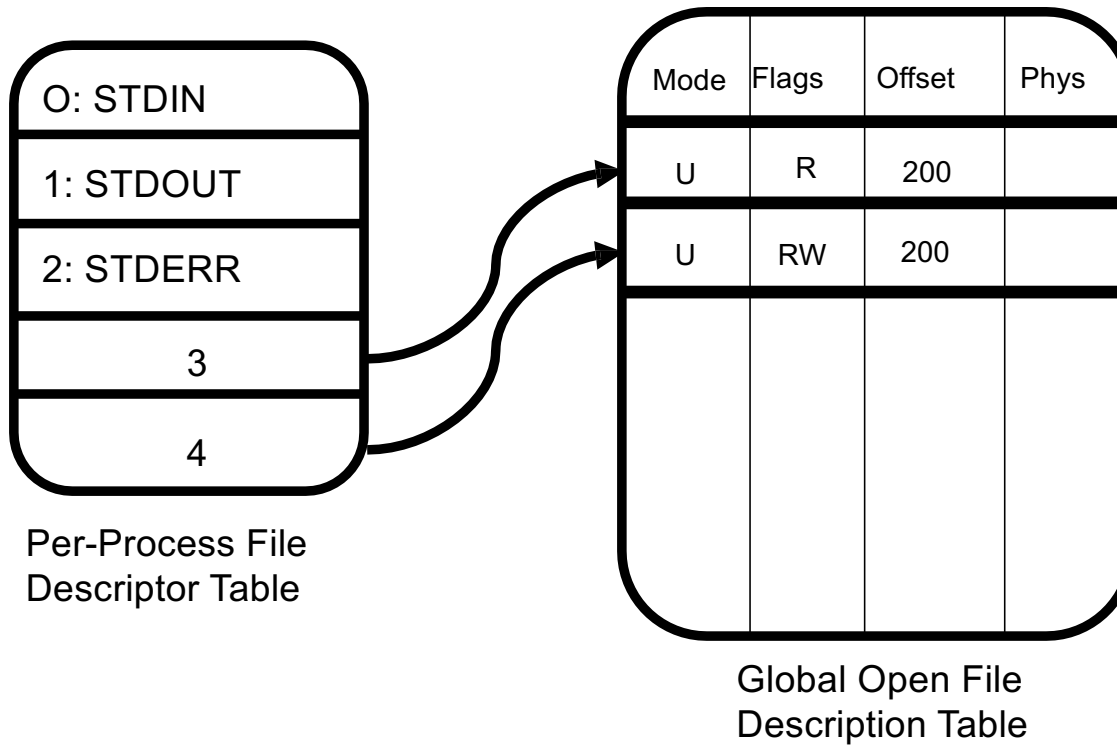
Disk management

- Basic entities on a disk:
 - **File**: user-visible group of blocks arranged sequentially in logical space
 - **Directory**: user-visible index mapping names to files
- The disk is accessed as linear array of sectors
- Old: Physical Position [cylinder, surface, sector]
- **New: Logical Block Addressing (LBA)**
 - Every sector has integer address
 - Controller translates from address $\Rightarrow$ physical position
 - Shields OS from structure of disk

What does the file system need?

- Track free disk blocks
 - Need to know where to put newly written data
- Track which blocks contain data for which files
 - Need to know where to read a file from
- Track files in a directory
 - Find list of file's blocks given its name
- Where do we maintain all of this?
 - Somewhere on disk

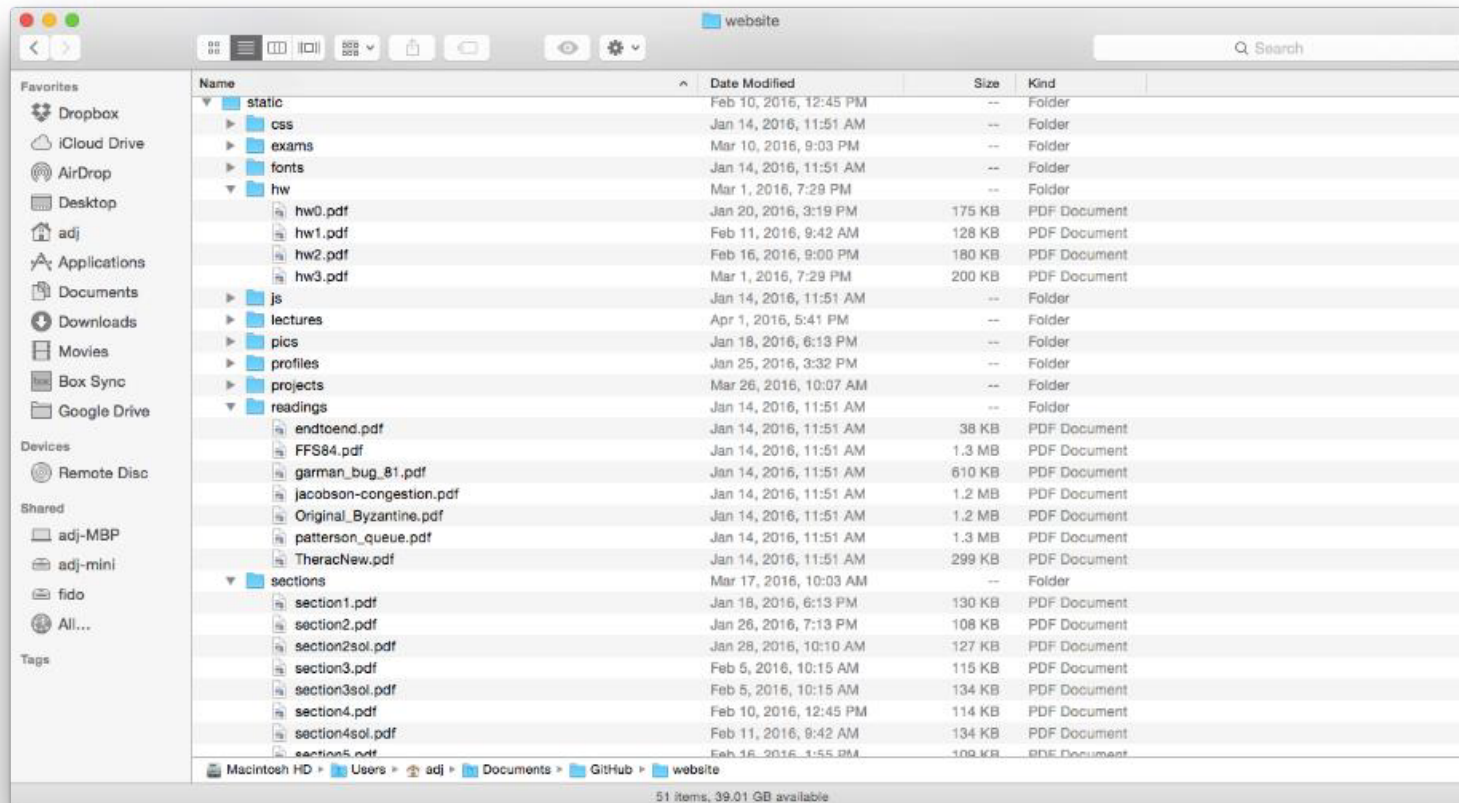
Recall: File Descriptors



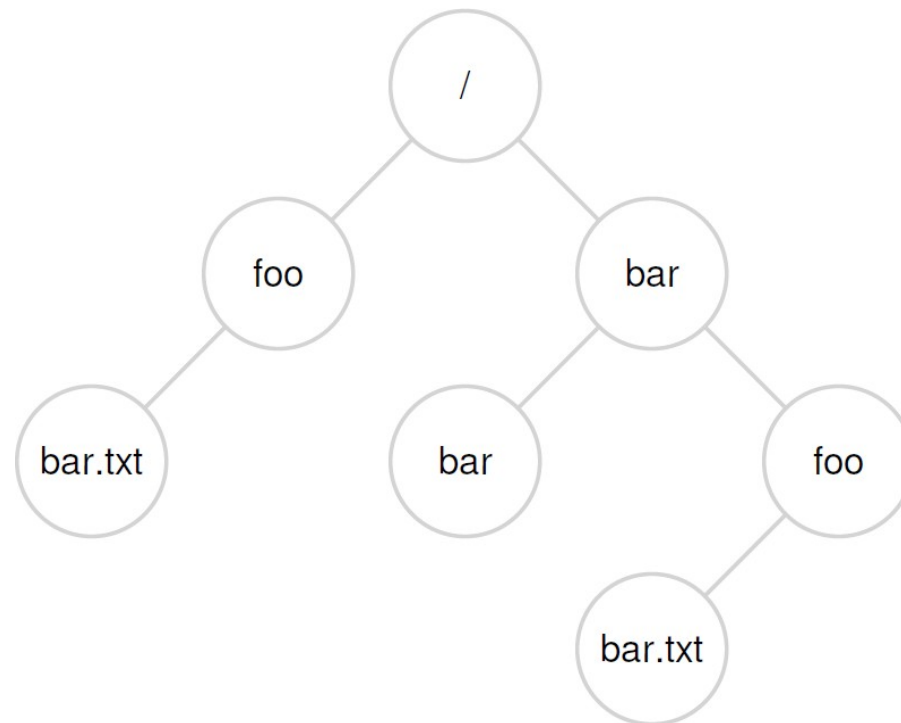
Critical factors in file system design

- (Hard) disk performance !!!
- Open before Read/Write
- Size is determined as they are used !!!
- Organized into directories
- Need to carefully allocate / free blocks

Files & Directories

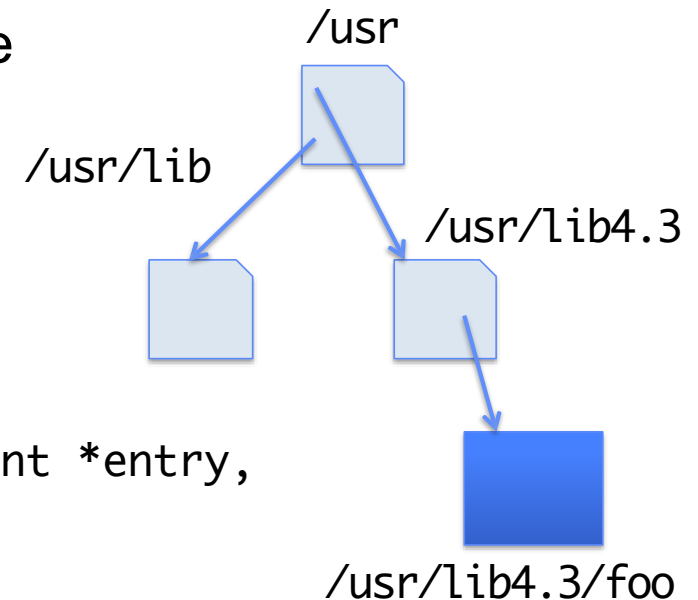


Files & Directories



Manipulating directories

- System calls to access directories
 - open/creat/readdir traverse the structure
 - mkdir/rmdir add/remove entries
 - link/unlink (rm)
- libc support
 - `DIR * opendir (const char *dirname)`
 - `struct dirent * readdir (DIR *dirstream)`
 - `int readdir_r (DIR *dirstream, struct dirent *entry, struct dirent **result)`

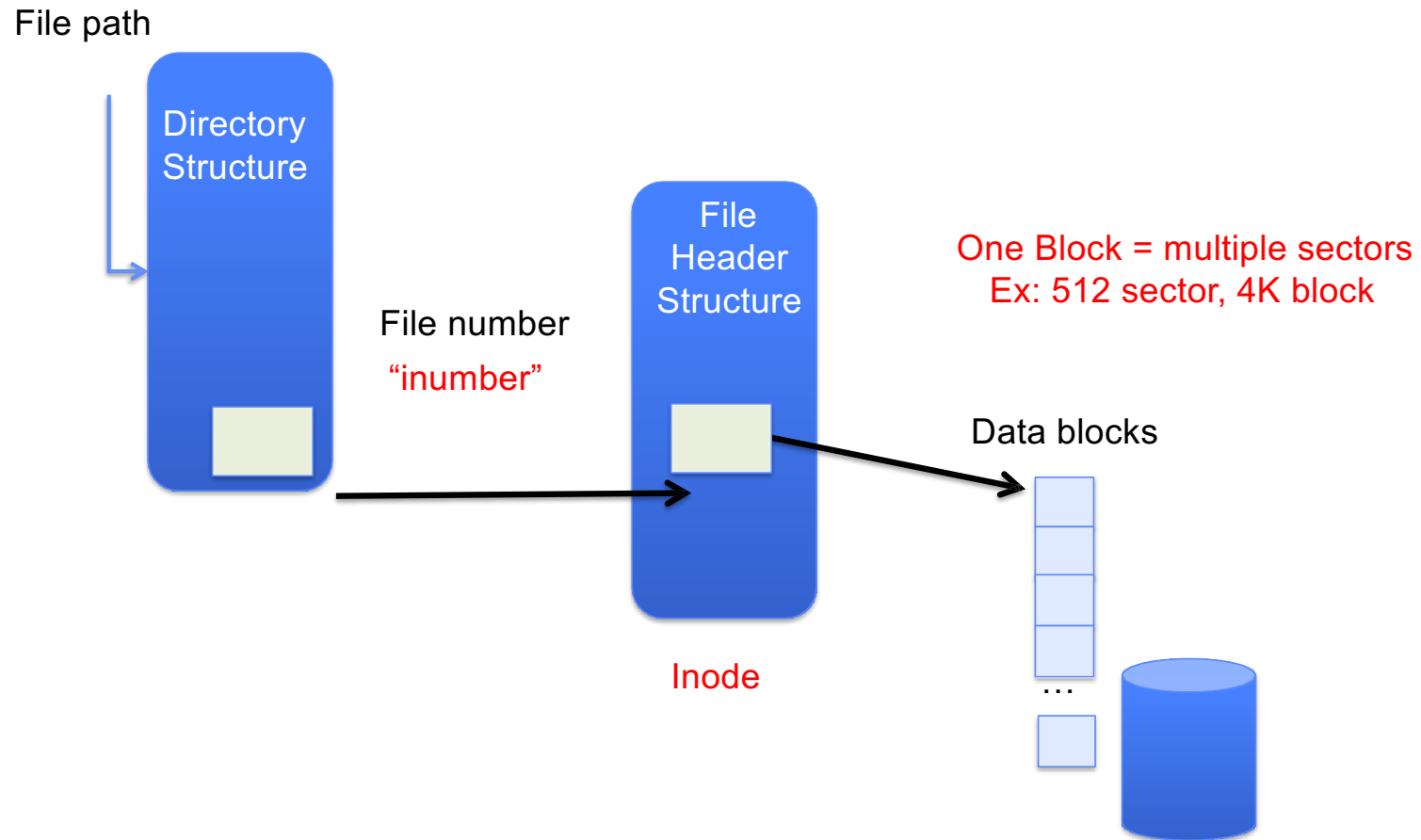


Components of a file system

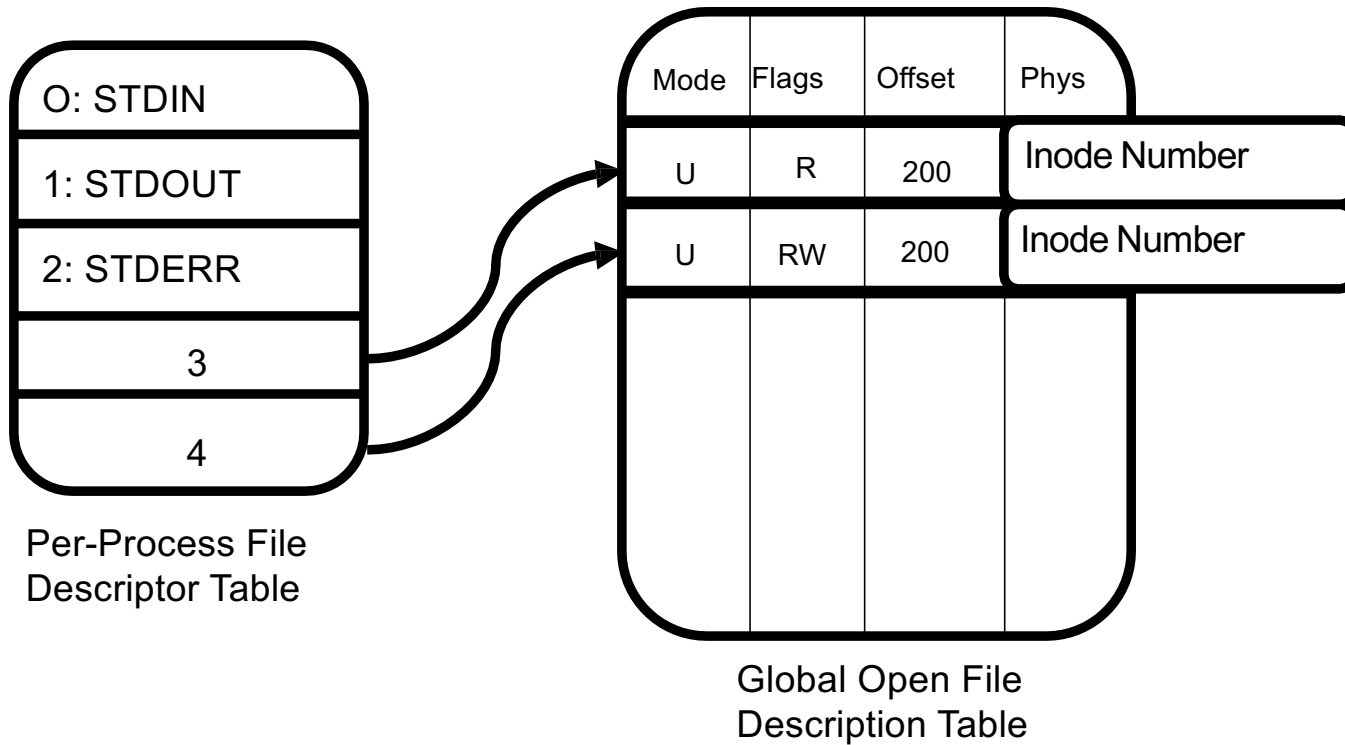
- **Superblock object:** information about file system
- **Free bitmaps:** what is allocated/not allocated
- **Inode object:** represents a specific file
- **Dentry object:** directory entry, single component of a path
- **File object:** open file associated with a process.
- **Blocks:** How files are stored on disk

Components of a file system

```
open (/laptop/xiao/os/foo.txt)
```



The (In)famous inode (index node)



How to get the inode number?

- Look up in **directory structure**
- Directory is a specialised file containing
 <file_name : inode number> mappings
- File number could be a file or another directory
- Each <file_name : inode> mapping is called a **directory entry**

How to read a file from disk

Let's read file /foo/bar.txt (Time goes downwards)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
open(bar)			read		read	read				
read()					read			read		
read()					read					
read()					write					
read()					read					
read()					write					
read()					read					
read()					write					

Thanks